

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4-ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	CH. Durga Prasad	Reg. No.:	192425305
Section / Batch:	2024	Date:	19.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Q8. Huffman Coding (Code Generation)

Given a set of characters and their respective frequencies, design a prefix-free binary code such that no code is a prefix of another. This ensures efficient and unambiguous decoding. Use Huffman Coding to generate valid binary codes for each character.

Code of Conduct

I CH. Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ch. Durga Prasad

RUBRICS FOR CALCULATION**Coding Challenge Task**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				

Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Aim:

To implement Huffman Coding in Python to generate efficient prefix-free binary codes for characters based on their frequencies.

Objectives:

1. To understand the basic concept of Huffman Coding.
2. To construct a Huffman Tree using character frequencies.
3. To assign binary codes to each character.
4. To ensure that the generated codes are prefix-free for unambiguous decoding.
5. To demonstrate the encoding process using the generated Huffman codes.
6. To reduce the overall number of bits required for data representation.
7. To analyze how character frequencies affect the efficiency of compression.

Pseudo code:

BEGIN

 DISPLAY "HUFFMAN CODING - LIVE DEMONSTRATION"

 READ number of characters

 FOR each character

 READ character

 READ frequency

 STORE character and frequency

 END FOR

 DISPLAY all characters and frequencies

 CREATE empty priority queue

 FOR each character

 CREATE a node containing character and frequency

 INSERT node into priority queue

 END FOR

 DISPLAY priority queue

 WHILE priority queue contains more than one node

 REMOVE two nodes with smallest frequencies

```

    DISPLAY the two selected nodes
    ADD their frequencies
    CREATE a new node with the combined frequency
    MAKE first node the LEFT child
    MAKE second node the RIGHT child
    INSERT new node into priority queue
    DISPLAY updated priority queue
END WHILE

SET remaining node as ROOT
SET LEFT branch = 0
SET RIGHT branch = 1
CALL GenerateCodes(ROOT, "")
PROCEDURE GenerateCodes(node, code)
    IF node is a leaf node THEN
        STORE code for the character
        RETURN
    END IF
    GenerateCodes(left child, code + "0")
    GenerateCodes(right child, code + "1")
END PROCEDURE

DISPLAY Huffman codes for all characters
READ message from user

SET encoded message = empty
FOR each character in message
    FIND Huffman code of character
    DISPLAY character and its code
    ADD code to encoded message
END FOR

DISPLAY original message
DISPLAY encoded message

```

DISPLAY "Huffman Coding completed successfully"

DISPLAY "Generated codes are prefix-free"

END

Step-by-Step Algorithm – Huffman Coding

1. **Start** the program.
2. **Read the number of characters** from the user.
3. For each character, **read its frequency** and store the character-frequency pairs.
4. **Display the input data** containing all characters and their frequencies.
5. Create an empty **priority queue**.
6. Create a node for each character and insert all nodes into the priority queue according to their frequencies.
7. Display the initial **priority queue**.
8. Select and remove the **two nodes with the smallest frequencies** from the priority queue.
9. **Combine the two frequencies** to create a new node with their sum as its frequency.
10. Make the first selected node the **left child** and the second selected node the **right child** of the new node.
11. Insert the new combined node back into the **priority queue**.
12. Repeat Steps **8–11** until only one node remains. This node becomes the **root of the Huffman Tree**.
13. Assign **0 to the left branch** and **1 to the right branch** of the Huffman Tree.
14. Traverse the Huffman Tree from the root to each character and generate its **binary Huffman code**.
15. Display the generated **Huffman codes** for all characters.
16. Read a **message from the user** containing the given characters.
17. For each character in the message, find its corresponding **Huffman code**.
18. Combine all the Huffman codes to form the **encoded message**.
19. Display the **original message**, individual character codes, and the final encoded message.
20. Display the **successful completion message** and confirm that the generated codes are prefix-free.
21. **Stop** the program.

Output:

```
=====
                        HUFFMAN CODING
                        LIVE DEMONSTRATION
=====

STEP 1: ENTER CHARACTERS AND FREQUENCIES
-----
Enter number of characters: 4
Enter character 1: A
Enter frequency of A: 11
Enter character 2: B
Enter frequency of B: 12
Enter character 3: C
Enter frequency of C: 13
Enter character 4: D
Enter frequency of D: 14

INPUT DATA
-----
A -> 11
B -> 12
C -> 13
D -> 14


STEP 2: CREATE PRIORITY QUEUE
-----
Priority Queue: A(11) B(12) C(13) D(14)


STEP 3: BUILD HUFFMAN TREE
-----

Step 1
Smallest nodes selected:
A = 11
B = 12
Combine: 11 + 12 = 23
After combining:
Priority Queue: C(13) D(14) NODE(23)

Step 2
Smallest nodes selected:
C = 13
D = 14
Combine: 13 + 14 = 27
After combining:
Priority Queue: NODE(23) NODE(27)

Step 3
Smallest nodes selected:
NODE = 23
NODE = 27
Combine: 23 + 27 = 50
After combining:
Priority Queue: NODE(50)
```

STEP 4: GENERATE HUFFMAN CODES

Binary Tree Rule:

Left = 0

Right = 1

Generated Codes:

A -> 00

B -> 01

C -> 10

D -> 11

STEP 5: ENCODE A MESSAGE

Enter a message using the given characters: ABCD

Encoding Process:

A -> 00

B -> 01

C -> 10

D -> 11

Original Message : ABCD

Encoded Message : 00011011

Result:

The Huffman Coding algorithm was successfully implemented in Python. The characters were assigned valid prefix-free binary codes based on their frequencies, and the message was successfully encoded using the generated Huffman codes.